

QA Automation Workshop — Theoretical Study Material

Hours 1–5: Foundations of Software Testing & Quality Assurance

For BSc CSIT Students — From Zero to Industry-Ready

About This Document

This study material provides the complete theoretical foundation for Hours 1 through 5 of the 16-hour QA Automation Workshop. Every concept is explained from first principles, assuming no prior knowledge of software testing. The material is designed to prepare you for practical workshop sessions, internship interviews, and entry-level QA roles.

How to Use This Material: Read each section sequentially before attending the corresponding workshop hour. Concepts build on each other — skipping ahead may cause confusion. Pay special attention to the "Interview Corner" and "Common Mistakes" sections, as these directly reflect what hiring managers test for.

Table of Contents

- [Hour 1 — Introduction to Software Testing & Quality Assurance](#)
 - [1.1 What Is Software Testing?](#)
 - [1.2 Why Does Software Testing Exist?](#)
 - [1.3 Quality Assurance vs Quality Control vs Testing](#)
 - [1.4 The Cost of Defects](#)
 - [1.5 Seven Principles of Software Testing](#)
 - [1.6 Who Is a Software Tester?](#)
 - [1.7 Hour 1 Interview Corner](#)
- [Hour 2 — SDLC vs STLC](#)
 - [2.1 What Is the Software Development Life Cycle \(SDLC\)?](#)
 - [2.2 SDLC Models](#)
 - [2.3 What Is the Software Testing Life Cycle \(STLC\)?](#)
 - [2.4 SDLC vs STLC — A Detailed Comparison](#)

- [2.5 How STLC Maps onto Agile](#)
 - [2.6 Hour 2 Interview Corner](#)
- [Hour 3 — Types of Testing](#)
 - [3.1 The Testing Landscape](#)
 - [3.2 Functional Testing Types](#)
 - [3.3 Non-Functional Testing Types](#)
 - [3.4 Testing Based on Knowledge of Code](#)
 - [3.5 Testing Based on Approach](#)
 - [3.6 Master Comparison Table](#)
 - [3.7 Hour 3 Interview Corner](#)
- [Hour 4 — Test Case Design Techniques](#)
 - [4.1 What Are Test Case Design Techniques?](#)
 - [4.2 Equivalence Partitioning](#)
 - [4.3 Boundary Value Analysis](#)
 - [4.4 Decision Table Testing](#)
 - [4.5 State Transition Testing](#)
 - [4.6 Error Guessing](#)
 - [4.7 Hour 4 Interview Corner](#)
- [Hour 5 — Writing Test Cases, Documentation & Bug Reporting](#)
 - [5.1 Anatomy of a Test Case](#)
 - [5.2 Characteristics of a Good Test Case](#)
 - [5.3 Test Case Documentation Best Practices](#)
 - [5.4 Bug Reporting](#)
 - [5.5 Severity vs Priority](#)
 - [5.6 The Defect Life Cycle](#)
 - [5.7 Hour 5 Interview Corner](#)

- [Consolidated Quick-Reference Glossary](#)
-

Hour 1 — Introduction to Software Testing & Quality Assurance

1.1 What Is Software Testing?

Imagine you have just finished building a house. Before you move in, would you not check whether the doors open properly, the plumbing works, the electrical wiring is safe, and the roof does not leak? Of course you would. Software testing is exactly this — checking that the software "house" you built actually works correctly, safely, and as the people who asked for it expected.

Definition

Software Testing is the process of evaluating a software application to detect differences between the expected behavior (what the software *should* do, according to requirements) and the actual behavior (what the software *actually* does when you use it). The goal is to find defects before the software reaches end users.

In professional language, software testing is a verification and validation activity that assesses the quality attributes of a software product — its correctness, completeness, reliability, security, usability, and performance.

A simple mental model:

Requirements (what we WANT) → Software (what we BUILT) → Testing (do they MATCH?)

If they match, the software is working correctly. If they do not match, we have found a **defect** (also called a **bug**).

Why It Matters

Every piece of software you use daily — Instagram, Google Maps, your banking app, Netflix — has been tested by teams of professional testers before it reached your phone. When that testing is incomplete or poorly done, real problems occur: money is lost, data is compromised, and in extreme cases (medical devices, aviation software), lives can be at risk.

Where It Is Used

Software testing is used in every industry where software exists, including e-commerce (Amazon, Flipkart), social media (Facebook, Instagram, TikTok), banking and finance (online banking, stock trading platforms), healthcare (hospital management systems,

diagnostic software), transportation (Uber, airline booking systems), gaming, government systems, and more.

1.2 Why Does Software Testing Exist?

Software testing exists because **humans make mistakes**. That is the fundamental, inescapable reason. Developers — no matter how skilled — are human beings writing complex instructions (code) that must handle millions of possible user actions, data combinations, and environmental conditions. Mistakes are inevitable.

Here are the specific reasons why testing is not optional but essential:

1. Human Error (Mistakes in Thinking) A developer might misunderstand a requirement. For example, a banking requirement says "transfer limit is \$10,000 per day." The developer might code it as \$10,000 per transaction instead of per day — a subtle but dangerous difference.

2. Complexity Modern software systems are extraordinarily complex. A single feature on Amazon (say, "Add to Cart") involves the user interface, the product database, the pricing engine, the inventory system, the recommendation engine, tax calculations, and more. A change in any one of these can break the feature in unexpected ways.

3. Changing Requirements In real projects, requirements change constantly. A client might say, "We also want to support cryptocurrency payments now." Every change introduces the risk of new defects and the risk of breaking existing features that were previously working (this is called a **regression**).

4. Communication Gaps Software is built by teams — product managers, designers, frontend developers, backend developers, database engineers. Information passes through many hands, and at each handoff, details can be lost or misunderstood, like a game of telephone.

5. Environmental Variability Your application might work perfectly on Chrome but crash on Safari. It might work on a fast Wi-Fi connection but fail on a slow 3G network. It might work on an iPhone 15 but not on an older Android phone. Testing must cover these variations.

6. Security Threats Hackers constantly probe for weaknesses. If a login form does not properly validate input, an attacker could inject malicious code (called SQL injection) and steal user data. Testing helps identify such vulnerabilities before attackers find them.

Real-World Example: The Therac-25 Radiation Machine

In the 1980s, a radiation therapy machine called Therac-25 had a software bug that caused it to deliver lethal doses of radiation to patients. Six patients were severely injured and three died. The root cause was a race condition in the software — a type of bug that adequate testing could have caught. This is one of the most cited examples in software engineering of why testing is not a luxury but a necessity.

Real-World Example: Knight Capital Group (2012)

A stock trading firm called Knight Capital deployed untested code to their production servers. Within 45 minutes, a software bug caused them to make millions of unintended trades, resulting in a loss of \$440 million. The company nearly went bankrupt overnight. The cause? Insufficient testing of a code deployment.

1.3 Quality Assurance vs Quality Control vs Testing

These three terms are frequently confused, even by professionals. Understanding the difference is essential for interviews and for understanding your role on a team.

Quality Assurance (QA)

Definition

Quality Assurance (QA) is a **process-oriented** discipline focused on **preventing** defects. QA asks: "Are we following the right processes and standards so that defects are less likely to be introduced in the first place?"

QA is proactive. It does not wait for bugs to appear — it puts processes in place to reduce the chance of bugs being created. Examples of QA activities include defining coding standards that all developers must follow, establishing a mandatory code review process (every piece of code must be reviewed by another developer before being merged), creating checklists for requirements gathering to ensure nothing is missed, and setting up automated build pipelines that run tests every time code is committed.

Analogy: QA is like a city implementing building codes and safety regulations *before* any construction begins. The goal is to prevent structural problems from ever occurring.

Quality Control (QC)

Definition

Quality Control (QC) is a **product-oriented** discipline focused on **detecting** defects in the actual product. QC asks: "Does this specific product meet the required quality standards?"

QC is reactive. It examines the product that has been built and checks whether it meets quality criteria. Examples of QC activities include reviewing completed code for defects, inspecting a finished user interface for visual bugs, running the software and checking whether features work as specified, and verifying that a delivered product matches the acceptance criteria.

Analogy: QC is like a building inspector who comes to examine a completed house and checks whether it meets all building codes and standards.

Testing

Definition

Software Testing is a **subset of QC**. It is the specific activity of executing the software (or examining it) to find defects. Testing is one of several QC activities but is often the most visible and time-consuming one.

Analogy: Testing is like turning on every faucet, flipping every light switch, and opening every door in that house to verify everything works.

Comparison Table

Aspect	Quality Assurance (QA)	Quality Control (QC)	Testing
Focus	Process	Product	Product (execution)
Nature	Preventive (proactive)	Detective (reactive)	Detective (reactive)
Goal	Prevent defects from being introduced	Find defects in the product	Find defects by executing software
Scope	Entire development process	Finished or in-progress product	Specific builds or features
Who does it	Everyone on the team	QC team, reviewers	Testers, QA engineers
Example	Defining coding standards	Code review, inspections	Running test cases, exploratory testing
Timing	Throughout the project	After development work is done	After a build is ready for testing

A["Software Quality Management"] --> B["Quality Assurance (QA)"]

A --> C["Quality Control (QC)"]

C --> D["Software Testing"]

C --> E["Code Reviews"]

C --> F["Inspections"]

C --> G["Walkthroughs"]

B --> H["Process Standards"]

B --> I["Training"]

B --> J["Audits"]

B --> K["Process Improvement"]

style A fill:#1a365d,stroke:#2d3748,color:#fff

style B fill:#065a82,stroke:#2d3748,color:#fff

style C fill:#065a82,stroke:#2d3748,color:#fff

style D fill:#e8871e,stroke:#2d3748,color:#fff

Common Beginner Mistake

Many beginners (and even some job listings) use "QA" and "Testing" interchangeably. They are not the same thing. In an interview, if asked "What is the difference between QA and Testing?" you must explain that QA is about prevention through processes while Testing is about detection through execution. Conflating them signals a lack of foundational understanding.

1.4 The Cost of Defects

One of the most important concepts in software testing is that **the later a defect is found, the more expensive it is to fix**. This principle, often called the "Cost of Quality" or illustrated by the "Defect Cost Curve," is the economic justification for investing in testing early and continuously.

The Relative Cost Multiplier

Phase Where Defect Is Found	Relative Cost to Fix	Real-World Meaning
Requirements gathering	1x (baseline)	A misunderstood requirement caught in a meeting costs almost nothing — just a conversation and a document update
Design phase	5x	A design flaw found during architecture review requires redesigning part of the system
Coding (development) phase	10x	A coding bug found during development requires debugging, fixing, retesting, and code review
Testing phase	20x	A bug found during testing requires a bug report, developer investigation, code fix, code review, rebuild, retest, and regression testing
Production (after release)	100x or more	A bug found by real users requires emergency triage, hotfix development, expedited testing, deployment, customer communication, potential legal liability, and reputation damage

Real-World Example: Samsung Galaxy Note 7 (2016)

Samsung released the Galaxy Note 7 with a battery defect that caused phones to catch fire. The initial recall and replacement program cost Samsung an estimated \$5.3 billion. The defect could have been caught during quality testing for a tiny fraction of that cost. This is the "100x cost" of finding a defect in production, taken to an extreme.

Why This Matters for Testers

This cost curve is the reason why modern software engineering emphasizes:

- **Shift-left testing** — starting testing activities as early as possible (even during requirements gathering) rather than waiting until after all code is written
- **Continuous testing** — running automated tests every time a developer commits code, catching defects within minutes rather than weeks
- **Test-Driven Development (TDD)** — writing test cases *before* writing the code, so defects are caught at the moment of creation

graph LR

A["Requirements
Cost: 1x"] --> B["Design
Cost: 5x"]

B --> C["Development
Cost: 10x"]

C --> D["Testing
Cost: 20x"]

D --> E["Production
Cost: 100x+"]

style A fill:#22c55e,stroke:#333,color:#fff

style B fill:#84cc16,stroke:#333,color:#000

style C fill:#eab308,stroke:#333,color:#000

style D fill:#f97316,stroke:#333,color:#fff

style E fill:#ef4444,stroke:#333,color:#fff

Industry Best Practice

In Agile teams at companies like Google and Netflix, automated tests run on every single code commit (a practice called Continuous Integration/Continuous Delivery, or CI/CD). If any test fails, the code commit is automatically rejected. This "shift-left" approach catches defects at the cheapest possible point — the moment they are created — rather than waiting for a separate testing phase.

1.5 Seven Principles of Software Testing

The International Software Testing Qualifications Board (ISTQB) — the global standard-setting body for software testing certification — defines seven foundational principles that every tester must understand. These principles are based on decades of industry experience and are among the most frequently asked interview topics.

Principle 1: Testing Shows the Presence of Defects, Not Their Absence

Testing can prove that defects exist, but it can never prove that a piece of software is completely defect-free. Even if you run a million tests and all pass, there could still be a defect lurking in an untested scenario.

Why this matters: This principle sets realistic expectations. When a stakeholder asks "Is this software bug-free?", the honest, professional answer is: "All tests we have executed have passed, and we have high confidence in the tested scenarios. However, testing cannot guarantee the absence of all defects."

Real-world example: Facebook has thousands of engineers and millions of automated tests, yet new bugs are still discovered daily. Testing reduces risk — it does not eliminate it.

Principle 2: Exhaustive Testing Is Impossible

For any non-trivial software, testing every possible combination of inputs, preconditions, and paths through the system is mathematically impossible. Consider a simple login form with a username field (up to 50 characters) and a password field (up to 30 characters). The number of possible input combinations (considering all possible characters at every position) is astronomically large — far more than could be tested in a human lifetime.

Why this matters: Since we cannot test everything, we must be strategic. We use techniques like risk-based testing (testing the riskiest features most thoroughly) and the test case design techniques you will learn in Hour 4 (Equivalence Partitioning, Boundary Value Analysis) to select a manageable number of test cases that provide maximum coverage.

Real-world example: Amazon's product search feature can receive literally billions of different search queries. Amazon's testers cannot test every possible query, so they test representative categories (common queries, empty queries, very long queries, queries with special characters, queries in different languages, etc.).

Principle 3: Early Testing Saves Time and Money

This directly follows from the "Cost of Defects" section above. Testing activities should start as early as possible in the development life cycle and should be integrated into every phase — not saved for the end.

In Agile teams, this means testers participate in sprint planning, review user stories for testability, and begin writing test cases before development starts. In traditional models, this means testers review requirements documents and design documents, not just the finished product.

Principle 4: Defects Cluster Together

In practice, a small number of modules or features contain the majority of defects. This is sometimes called the **Pareto Principle** (or 80/20 rule) applied to testing: roughly 80% of bugs are found in 20% of the modules.

Why this matters: If you find several bugs in one module, you should test that module more thoroughly — there are likely more bugs hiding there. This also helps in risk-based test prioritization: focus your limited testing time on the modules with the highest historical defect density.

Real-world example: In a typical e-commerce application, the checkout and payment module tends to have the highest defect density because it involves the most complex business logic (pricing, discounts, taxes, payment gateway integration, inventory updates, order confirmation). Experienced testers allocate more testing time to this module than to, say, the "About Us" page.

Principle 5: The Pesticide Paradox

If the same tests are run over and over again, they will eventually stop finding new defects — just as pests develop resistance to a pesticide that is used repeatedly. The software does not change, so the same tests keep passing.

Why this matters: Test cases must be regularly reviewed, updated, and expanded. New test cases should be written to cover previously untested scenarios, edge cases, and new features. Simply re-running old test suites without enhancement gives a false sense of security.

Real-world example: A QA team at a ride-sharing company initially wrote tests for "request a ride in your current city." After six months of these tests always passing, a new bug appeared: rides failed when the user was at an airport near a city border (the GPS location mapped to a different city). The existing tests never caught this because they never tested the "edge" scenario of city boundaries. This is the Pesticide Paradox in action — the tests became "resistant" to new types of defects.

Principle 6: Testing Is Context-Dependent

There is no single "right way" to test software. The testing approach depends entirely on the context — what type of software is being tested, who uses it, what the risks are, and what industry regulations apply.

Why this matters: Testing a children's educational game is fundamentally different from testing banking software or medical device firmware. A defect in a game might cause a minor annoyance; a defect in a bank transfer could lose someone's money; a defect in a pacemaker could end someone's life. The rigor, depth, and techniques used must match the context.

Context	Testing Emphasis
Banking application	Security testing, data integrity, regulatory compliance, transaction accuracy

Context	Testing Emphasis
E-commerce (Amazon)	Performance under high traffic, payment processing, search accuracy, mobile responsiveness
Social media (Instagram)	Usability, image/video handling, notification delivery, content moderation
Medical device software	Safety-critical testing, regulatory standards (FDA), extensive documentation
Mobile gaming	Performance across devices, in-app purchases, offline behavior, battery consumption

Principle 7: The Absence-of-Errors Fallacy

Finding and fixing defects does not help if the system built is fundamentally wrong — if it does not meet user needs or does not fulfill business requirements. A perfectly bug-free application that nobody wants to use is a failed product.

Why this matters: Testing must verify not just that the software works correctly (verification: "Are we building the product right?") but also that it is the right product (validation: "Are we building the right product?").

Real-world example: Google Wave (2009) was a well-engineered, relatively bug-free communication tool. But users found it confusing and unnecessary — it did not solve a real problem better than existing tools. Despite being technically sound, it was shut down because it failed the validation test: it was not the product users actually needed.

1.6 Who Is a Software Tester?

Understanding the career landscape in software testing will help you see where this workshop is taking you and what opportunities are available.

Roles in the Testing Ecosystem

Role	Primary Responsibilities	Typical Skills
Manual Tester / QA Analyst	Writes and executes test cases manually, reports bugs, verifies fixes	Domain knowledge, attention to detail, communication, test case design

Role	Primary Responsibilities	Typical Skills
Automation Test Engineer	Writes automated test scripts using tools and frameworks, maintains test suites	Programming (Java, Python, JavaScript), Selenium, API testing tools
SDET (Software Development Engineer in Test)	Builds test frameworks, test infrastructure, and tooling; writes complex automation	Strong programming, system design, CI/CD, DevOps
Performance Test Engineer	Designs and executes performance, load, and stress tests	JMeter, Gatling, performance analysis, server monitoring
Security Tester / Penetration Tester	Identifies security vulnerabilities in applications	OWASP knowledge, security tools (Burp Suite), ethical hacking
QA Lead / Test Manager	Plans testing strategy, manages the QA team, reports to stakeholders	Leadership, planning, risk assessment, metrics

Career Path Insight

The industry is rapidly moving toward automation. While manual testing skills are essential as a foundation (you must understand *what* to test before you can automate *how* to test), the highest-demand, highest-paying QA roles require programming and automation skills. This workshop is designed to give you both: manual testing fundamentals in the first five hours, followed by automation skills (Postman, JMeter, Selenium) in the remaining eleven hours.

The Tester Mindset

The most important quality of a good tester is not technical skill — it is **mindset**. A tester's mindset is fundamentally different from a developer's mindset.

A developer thinks constructively: "How do I build this feature so it works?" A tester thinks destructively (in a good way): "How can I break this feature? What can go wrong? What did the developer not think of?"

This does not mean testers are adversarial. It means testers are the safety net — the last line of defense between a buggy product and the user. A good tester is curious ("What

happens if I do this?"), skeptical ("Does this really work in all cases?"), detail-oriented ("The spec says the button should be blue, but this is teal"), and communicative ("Here is exactly what went wrong, how to reproduce it, and why it matters").

1.7 Hour 1 Interview Corner

Frequently Asked Interview Questions — Hour 1 Topics

Q1: What is software testing? Model Answer: Software testing is the process of evaluating a software application to identify differences between expected and actual behavior. It involves executing the application under controlled conditions, observing the results, and comparing them to specified requirements. The goal is to find defects, ensure the software meets quality standards, and provide confidence that the product is fit for use.

Q2: What is the difference between QA, QC, and Testing? Model Answer: Quality Assurance is a process-oriented approach focused on preventing defects by establishing standards and processes throughout the development lifecycle. Quality Control is a product-oriented approach focused on identifying defects in the finished or in-progress product through activities like reviews and inspections. Testing is a subset of Quality Control — it is the specific activity of executing the software to find defects.

Q3: What are the seven principles of software testing? Explain any two. Model Answer: The seven principles are: (1) Testing shows presence of defects, not absence, (2) Exhaustive testing is impossible, (3) Early testing saves time and money, (4) Defects cluster together, (5) The Pesticide Paradox, (6) Testing is context-dependent, (7) Absence-of-errors fallacy. *[Then explain two in detail using the material above.]*

Q4: Why can't we achieve 100% test coverage? Model Answer: Exhaustive testing is impossible because even simple software has an astronomically large number of possible input combinations, system states, and execution paths. Instead, we use risk-based approaches and test design techniques like Equivalence Partitioning and Boundary Value Analysis to select a representative set of test cases that maximize coverage within practical constraints.

Q5: What is the Pesticide Paradox? Model Answer: The Pesticide Paradox states that if the same tests are run repeatedly, they will eventually stop finding new defects. This happens because the tests only cover specific scenarios and the software is not changing. To counteract this, test cases must be regularly reviewed, updated, and new tests must be created for previously untested scenarios and edge cases.

Q6: Explain "shift-left testing." Model Answer: Shift-left testing is the practice of starting testing activities as early as possible in the software development lifecycle — ideally during requirements gathering — rather than waiting until after development is complete. The term comes from visualizing the development timeline from left (requirements) to right (production): testing "shifts" to the left side. This approach reduces defect cost because bugs found earlier are cheaper to fix.

Hour 2 — SDLC vs STLC

2.1 What Is the Software Development Life Cycle (SDLC)?

Before you can understand how testing fits into software development, you need to understand how software is developed in the first place.

Definition

The **Software Development Life Cycle (SDLC)** is a structured framework that defines the phases involved in developing a software application from initial concept to final deployment and maintenance. It provides a systematic approach to building software that meets quality standards and customer expectations.

Think of the SDLC as a recipe for building software. Just as a recipe has steps (gather ingredients, prepare, cook, serve), the SDLC has phases that teams follow to go from an idea to a working product.

The Core SDLC Phases

Every SDLC model, regardless of its specific approach, includes these fundamental activities:

Phase 1: Requirement Gathering and Analysis The team works with stakeholders (clients, users, business owners) to understand *what* the software should do. Business analysts document these needs as detailed requirements. For example, "Users should be able to log in using their email and password, or through Google OAuth."

Phase 2: System Design Architects and senior developers decide *how* to build what was specified. This includes choosing technologies (programming languages, databases, cloud services), defining the system architecture (monolith vs. microservices), designing the database schema, creating API contracts, and designing the user interface.

Phase 3: Implementation (Coding) Developers write the actual code that implements the design. This is typically the longest phase and involves frontend development, backend development, database implementation, and integration of various components.

Phase 4: Testing The completed (or partially completed) software is tested against the requirements to find defects. This is where QA engineers and testers are most visibly active, though — as we discussed in Hour 1 — testing activities ideally begin much earlier.

Phase 5: Deployment The tested software is released to production (made available to actual users). Deployment can be done all at once (a "big bang" release) or gradually (canary releases, blue-green deployments, feature flags).

Phase 6: Maintenance After deployment, the software requires ongoing maintenance: fixing newly discovered bugs, releasing updates, adding new features, and adapting to changes in the environment (new operating systems, new devices, new regulations).

graph LR

A["1. Requirements"] --> B["2. Design"]

B --> C["3. Implementation"]

C --> D["4. Testing"]

D --> E["5. Deployment"]

E --> F["6. Maintenance"]

F -. -> A

style A fill:#065a82,stroke:#333,color:#fff

style B fill:#1c7293,stroke:#333,color:#fff

style C fill:#065a82,stroke:#333,color:#fff

style D fill:#e8871e,stroke:#333,color:#fff

style E fill:#1c7293,stroke:#333,color:#fff

style F fill:#065a82,stroke:#333,color:#fff

2.2 SDLC Models

The SDLC phases listed above can be organized in different ways. Each organization is called a "model." Different models suit different types of projects. Understanding these models is essential because the model your team uses directly determines how and when you do testing.

Waterfall Model

The Waterfall model is the oldest and most linear SDLC model. Each phase must be fully completed before the next phase begins, and there is no going back to a previous phase (like water flowing downhill — it does not flow back up).

graph TD

A["Requirements"] --> B["Design"]

B --> C["Implementation"]

C --> D["Testing"]

D --> E["Deployment"]

E --> F["Maintenance"]

style A fill:#065a82,stroke:#333,color:#fff

style B fill:#1c7293,stroke:#333,color:#fff

style C fill:#065a82,stroke:#333,color:#fff

style D fill:#e8871e,stroke:#333,color:#fff

style E fill:#1c7293,stroke:#333,color:#fff

style F fill:#065a82,stroke:#333,color:#fff

How testing works in Waterfall: Testing happens only *after* all development is complete. Testers receive the finished product, test it, and report defects. This means defects are found late (expensive to fix) and there is significant delay between development and feedback.

When to use: Projects where requirements are very well understood, unlikely to change, and thoroughly documented upfront. Examples include government contracts with fixed specifications, embedded systems with regulatory requirements (medical devices, aviation).

Advantages: Simple to understand and manage, clear milestones, well-documented.

Limitations: Inflexible to requirement changes, testing happens very late, high risk of delivering a product the customer does not want (because there is no user feedback until the very end), long time before users see anything.

V-Model (Verification and Validation Model)

The V-Model is an extension of the Waterfall model where each development phase has a corresponding testing phase. The left side of the "V" represents development phases (going down), and the right side represents testing phases (going up). Each testing phase directly validates a specific development phase.

graph TD

A["Requirements Analysis"] ---|"Acceptance Test Planning"| H["User Acceptance Testing"]

B["System Design"] ---|"System Test Planning"| G["System Testing"]

C["Architecture Design"] ---|"Integration Test Planning"| F["Integration Testing"]

D["Module Design"] ---|"Unit Test Planning"| E["Unit Testing"]

D --> IMPL["Coding / Implementation"]

IMPL --> E

style A fill:#065a82,stroke:#333,color:#fff

style B fill:#1c7293,stroke:#333,color:#fff

style C fill:#065a82,stroke:#333,color:#fff

style D fill:#1c7293,stroke:#333,color:#fff

style IMPL fill:#e8871e,stroke:#333,color:#000

style E fill:#1c7293,stroke:#333,color:#fff

style F fill:#065a82,stroke:#333,color:#fff

style G fill:#1c7293,stroke:#333,color:#fff

style H fill:#065a82,stroke:#333,color:#fff

How testing works in the V-Model: Test planning and test case design begin at the same time as the corresponding development phase. When requirements are being written,

acceptance test cases are being planned. When the system is being designed, system test cases are being planned. This is an early form of "shift-left" testing.

When to use: Projects where requirements are clear and the relationship between requirements and tests must be explicitly traced (common in regulated industries — healthcare, finance, defense).

Advantages: Testing is planned early, clear traceability between development and testing phases, defects are found earlier than in pure Waterfall.

Limitations: Still sequential and inflexible, does not handle changing requirements well, still no working software until late in the process.

Agile Model

Agile is not a single model but a family of methodologies (Scrum, Kanban, XP, SAFe) that share common values defined in the Agile Manifesto (2001). Agile builds software incrementally — in short cycles called **iterations** or **sprints** (typically 1–4 weeks). Each sprint delivers a small, working, tested increment of the product.

graph LR

S1["Sprint 1
Plan → Build → Test → Release"] --> S2["Sprint 2
Plan → Build → Test → Release"]

S2 --> S3["Sprint 3
Plan → Build → Test → Release"]

S3 --> S4["Sprint N
Plan → Build → Test → Release"]

style S1 fill:#065a82,stroke:#333,color:#fff

style S2 fill:#1c7293,stroke:#333,color:#fff

style S3 fill:#065a82,stroke:#333,color:#fff

style S4 fill:#e8871e,stroke:#333,color:#fff

How testing works in Agile: Testing is not a separate phase — it is integrated into every sprint. Testers are part of the cross-functional team and participate from sprint planning through delivery. Testing happens continuously: unit tests are written during development, integration tests run in CI/CD pipelines, and manual/exploratory testing happens during the sprint. Each sprint ends with a potentially shippable product increment.

When to use: Most modern software projects use Agile, especially where requirements are expected to evolve, user feedback is valuable, and rapid delivery is important. This includes virtually all web applications, mobile apps, SaaS products, and startup projects.

Advantages: Highly flexible, delivers working software quickly, continuous user feedback, testing is integrated (not an afterthought), defects are found early and cheaply.

Limitations: Requires disciplined teams, less documentation (can be a problem for regulated industries), difficult to predict total timeline and cost upfront, can be chaotic if poorly implemented.

Comparison of SDLC Models

Aspect	Waterfall	V-Model	Agile
Approach	Sequential, linear	Sequential with parallel test planning	Iterative, incremental
Flexibility	Very low — changes are costly	Low — still sequential	Very high — embraces change
Testing phase	After all development is complete	Planned early, executed after development	Continuous, throughout every sprint
When tester joins	Late (after development)	Early (during planning), executes late	From the very beginning, every sprint
User feedback	Only after final delivery	Only after final delivery	Every sprint (every 1–4 weeks)
Risk of wrong product	High	High	Low (constant feedback corrects course)
Documentation	Heavy, detailed	Heavy, detailed	Light, just enough
Best for	Fixed, well-known requirements	Regulated, traceable requirements	Evolving requirements, fast delivery

Industry Reality Check

As of today, the vast majority of software companies (estimated 70–85%) use some form of Agile. Waterfall is primarily found in government, defense, and certain regulated industries.

If you are preparing for a career in software testing, you will almost certainly work in an Agile environment. This workshop assumes Agile as the primary context while noting where traditional models differ.

2.3 What Is the Software Testing Life Cycle (STLC)?

Now that you understand how software is *developed* (SDLC), let us focus on how software is *tested*.

Definition

The **Software Testing Life Cycle (STLC)** is a sequence of well-defined phases that are systematically executed during the testing process. It defines *what* testing activities to perform and *when* to perform them to ensure thorough and organized testing.

The STLC is to testing what the SDLC is to development: a structured framework that ensures nothing is missed and everything is done in the right order.

The Six Phases of STLC

graph LR

A["1. Requirement
Analysis"] --> B["2. Test
Planning"]

B --> C["3. Test Case
Development"]

C --> D["4. Environment
Setup"]

D --> E["5. Test
Execution"]

E --> F["6. Test Cycle
Closure"]

style A fill:#065a82,stroke:#333,color:#fff

style B fill:#1c7293,stroke:#333,color:#fff

style C fill:#065a82,stroke:#333,color:#fff

style D fill:#1c7293,stroke:#333,color:#fff

style E fill:#e8871e,stroke:#333,color:#fff

style F fill:#065a82,stroke:#333,color:#fff

Phase 1: Requirement Analysis

What happens: The QA team reviews the requirements documents (or user stories, in Agile) to understand *what* needs to be tested. They identify testable requirements, clarify ambiguities with business analysts and product owners, and determine the scope of testing.

Key activities:

- Reviewing the Software Requirements Specification (SRS) or user stories
- Identifying testable and non-testable requirements
- Creating a Requirement Traceability Matrix (RTM) that maps every requirement to its corresponding test cases
- Identifying the types of testing required (functional, performance, security, etc.)
- Raising questions about ambiguous or conflicting requirements

Deliverables: RTM (initial version), list of questions/clarifications, automation feasibility report

Real-world example: A requirement for a banking app states "The system should handle concurrent transactions efficiently." A good QA analyst would flag this as too vague — "efficiently" is not measurable. They would ask: "What is the expected response time? How many concurrent transactions should the system support? What happens when the limit is exceeded?" These clarifications prevent defects that would arise from misinterpretation.

Phase 2: Test Planning

What happens: The Test Lead or Test Manager creates a comprehensive test plan — a document that describes the testing strategy, objectives, scope, approach, resources, schedule, and deliverables.

Key activities:

- Defining the scope of testing (what will be tested and, importantly, what will *not* be tested)
- Selecting the testing approach and types of testing
- Estimating testing effort and timelines
- Identifying required tools (Selenium, Postman, JMeter, etc.)
- Defining entry criteria (conditions that must be met before testing can start) and exit criteria (conditions that must be met before testing can be considered complete)

- Identifying risks and creating mitigation plans
- Assigning roles and responsibilities

Deliverables: Test Plan document, Effort Estimation document

Definition

Entry Criteria are the conditions that must be met before a testing phase can begin.

Examples: "All development for Sprint 5 features is code-complete," "The test environment is set up and accessible," "Test data has been prepared."

Exit Criteria are the conditions that must be met before testing is considered complete.

Examples: "95% of test cases have been executed," "All critical and high-priority defects have been fixed and retested," "No open showstopper defects remain."

Phase 3: Test Case Development

What happens: Testers write detailed test cases and (if applicable) test scripts for automation. They also prepare test data — the specific data that will be used during testing (valid usernames, invalid passwords, sample products, test credit card numbers, etc.).

Key activities:

- Writing test cases using formal techniques (Equivalence Partitioning, Boundary Value Analysis, Decision Tables — covered in Hour 4)
- Reviewing test cases with peers (test case review)
- Preparing test data
- Writing automation scripts (if applicable)

Deliverables: Test Cases, Test Data, Automated Test Scripts (if applicable)

Phase 4: Test Environment Setup

What happens: The team sets up the hardware and software environment where testing will be performed. This environment should mirror the production environment as closely as possible.

Key activities:

- Setting up test servers, databases, and network configurations
- Installing required software and tools
- Configuring test environments (development, staging, pre-production)

- Performing a "smoke test" to verify the environment is functional

Deliverables: Test Environment setup documentation, Smoke Test Results

Real-world example: If Netflix's production environment runs on AWS with specific server configurations, the test environment should replicate that setup as closely as possible. Testing on a different configuration might miss bugs that only appear in the production setup.

Phase 5: Test Execution

What happens: Testers execute the test cases against the software build, comparing actual results to expected results. When discrepancies are found, defects are logged. Fixed defects are retested (re-verification) and the surrounding functionality is re-checked (regression testing).

Key activities:

- Executing test cases (manually or through automation tools)
- Logging defects in a defect tracking tool (Jira, Bugzilla, Azure DevOps)
- Mapping defects to test cases in the RTM
- Re-testing fixed defects
- Performing regression testing to ensure fixes did not break other things
- Updating test case status (Pass, Fail, Blocked, Not Executed)

Deliverables: Test Execution Results, Defect Reports, Updated RTM

Phase 6: Test Cycle Closure

What happens: Once exit criteria are met, the testing cycle is formally closed. The team evaluates what went well, what could be improved, and documents lessons learned for future projects.

Key activities:

- Evaluating exit criteria completion
- Preparing test summary reports (total tests executed, pass rate, defect density, defect severity distribution)
- Documenting lessons learned and best practices
- Archiving test artifacts (test cases, defect reports, test data) for future reference

- Providing sign-off for release

Deliverables: Test Closure Report, Test Metrics, Lessons Learned document

2.4 SDLC vs STLC — A Detailed Comparison

Aspect	SDLC	STLC
Full form	Software Development Life Cycle	Software Testing Life Cycle
Purpose	Defines how software is developed from concept to deployment	Defines how software is tested from requirement analysis to test closure
Scope	Covers the entire product development	Covers only the testing portion of development
Performed by	The entire development team (developers, designers, architects, PMs, testers)	Primarily the QA/testing team
Phases	Requirements → Design → Implementation → Testing → Deployment → Maintenance	Requirement Analysis → Test Planning → Test Case Development → Environment Setup → Test Execution → Test Closure
Focus	Building the right product correctly	Validating that the product works as expected
Relationship	STLC is a subset that runs within the SDLC	STLC operates within the broader SDLC framework
Output	Working software product	Tested, validated software with documented quality evidence

Key Insight

STLC is not separate from SDLC — it is embedded within it. In Waterfall, the STLC runs mostly during the "Testing" phase of the SDLC. In Agile, the STLC runs within *every sprint*, meaning all six STLC phases happen (in compressed form) within each 1–4 week sprint cycle.

2.5 How STLC Maps onto Agile

In an Agile team using Scrum, a typical sprint includes testing activities mapped to STLC phases as follows:

Sprint Activity	STLC Phase Equivalent
Sprint Planning (Day 1): tester reviews user stories, identifies test scenarios, asks clarification questions	Requirement Analysis
Sprint Planning / Early Sprint: tester estimates test effort, identifies risks, determines testing approach	Test Planning
During Sprint (Days 2–8): tester writes test cases, prepares test data, writes automation scripts	Test Case Development
CI/CD pipeline, staging environment configuration	Environment Setup
During Sprint (Days 3–10): tester executes tests, logs bugs, retests fixes, runs regression suite	Test Execution
Sprint Review / Retrospective: team reviews quality metrics, discusses testing improvements for next sprint	Test Cycle Closure

Industry Best Practice

In high-performing Agile teams, the tester does not wait until code is "finished" to start testing. They begin writing test cases as soon as user stories are finalized (often before development starts), and they start testing completed features while developers are still working on other features within the same sprint. This parallelism is key to completing testing within the sprint timebox.

2.6 Hour 2 Interview Corner

Q1: What is the difference between SDLC and STLC? Model Answer: SDLC is the overall framework for developing software, covering everything from requirements gathering to deployment and maintenance. STLC is specifically focused on the testing activities within that lifecycle. STLC is a subset that operates within the SDLC. The SDLC produces the software; the STLC validates that the software meets its requirements.

Q2: Explain the phases of STLC. Model Answer: The six phases are: (1) Requirement Analysis — understanding what needs to be tested, (2) Test Planning — defining the testing strategy and logistics, (3) Test Case Development — writing test cases and preparing test data, (4) Environment Setup — configuring the test infrastructure, (5) Test Execution — running tests and reporting defects, (6) Test Cycle Closure — evaluating results and documenting lessons learned. Each phase has defined entry criteria, activities, and deliverables.

Q3: What are entry and exit criteria? Give examples. Model Answer: Entry criteria are preconditions that must be met before a phase can begin. For example, test execution cannot begin until the test environment is ready and the build is deployed. Exit criteria are conditions that signal a phase is complete. For example, test execution is complete when 95% of planned test cases are executed, all critical bugs are fixed, and the defect leakage rate is below the acceptable threshold.

Q4: In Agile, when does testing start? Model Answer: In Agile, testing starts from Day 1 of the sprint. During sprint planning, testers review user stories, identify test scenarios, and raise clarification questions — this is the Requirement Analysis phase of the STLC happening at the very start. Test case writing begins as soon as stories are finalized, and test execution begins as soon as the first features are code-complete, running in parallel with ongoing development within the same sprint.

Q5: Compare Waterfall and Agile models from a tester's perspective. Model Answer: In Waterfall, testing is a distinct phase that occurs after all development is complete. Testers receive a fully built product, which means defects are found late and are expensive to fix. In Agile, testing is integrated into every sprint. Testers work alongside developers from the beginning, write test cases before or during development, and test features as they are completed. This continuous testing approach finds defects early (cheaper to fix) and provides faster feedback to the team.

Hour 3 — Types of Testing

3.1 The Testing Landscape

Software testing is not a single activity — it is a broad discipline with many specialized types, each designed to evaluate a different aspect or attribute of the software.

Understanding these types, when to use each one, and how they relate to each other is one of the most important skills a tester can have.

Why So Many Types?

Think of a car inspection. You do not just start the engine and declare the car "tested." You check the brakes (safety), measure acceleration (performance), test the fuel efficiency (economy), verify the airbags deploy correctly (safety under stress), check the paint job (appearance), test the door locks (security), verify the GPS works (functionality), and more. Each check evaluates a different quality attribute. Software testing works the same way — different types of tests evaluate different quality attributes of the software.

We will organize testing types into several categories:

graph TD

A["Software Testing"] --> B["Based on What You Test"]

A --> C["Based on Code Knowledge"]

A --> D["Based on Approach"]

B --> B1["Functional Testing"]

B --> B2["Non-Functional Testing"]

B1 --> B1a["Unit Testing"]

B1 --> B1b["Integration Testing"]

B1 --> B1c["System Testing"]

B1 --> B1d["UAT"]

B1 --> B1e["Smoke Testing"]

B1 --> B1f["Sanity Testing"]

B1 --> B1g["Regression Testing"]

B2 --> B2a["Performance Testing"]

B2 --> B2b["Security Testing"]

B2 --> B2c["Usability Testing"]

B2 --> B2d["Compatibility Testing"]

C --> C1["Black-Box Testing"]

C --> C2["White-Box Testing"]

C --> C3["Grey-Box Testing"]

D --> D1["Manual Testing"]

D --> D2["Automated Testing"]

style A fill:#1a365d,stroke:#333,color:#fff

style B fill:#065a82,stroke:#333,color:#fff

style C fill:#065a82,stroke:#333,color:#fff

style D fill:#065a82,stroke:#333,color:#fff

3.2 Functional Testing Types

Functional testing verifies that the software does what it is supposed to do — that each function or feature works according to the specified requirements. It answers the question: "Does this feature produce the correct output for a given input?"

Unit Testing

Definition

Unit Testing tests individual, isolated components (units) of the software — typically a single function, method, or class — to verify that each unit works correctly on its own.

Who does it: Developers (not typically QA testers). Unit testing is considered a development activity.

When it happens: During the coding phase, as each unit is being written. In Test-Driven Development (TDD), unit tests are written *before* the code.

Real-world example: A developer at Amazon writes a function called `calculateShippingCost(weight, destination, shippingMethod)`. A unit test would call this function with specific inputs (e.g., `weight=2kg`, `destination="New York"`, `method="Express"`)

and verify the output matches the expected cost. The test focuses on this one function in isolation, without involving the database, the UI, or any other system component.

Advantages: Extremely fast to run (milliseconds per test), catches bugs at the cheapest possible point, gives developers immediate feedback, serves as living documentation of how each function should behave.

Limitations: Only tests individual units in isolation — cannot catch problems that arise when units interact with each other (those are caught by integration testing).

Integration Testing

Definition

Integration Testing tests the interactions between two or more integrated units/modules to ensure they work together correctly. It verifies that data flows correctly between components and that integrated components produce the expected combined behavior.

Who does it: Developers and/or QA engineers.

When it happens: After unit testing, when individual modules are combined.

Real-world example: On an e-commerce site, after a user places an order, the order module must communicate with the payment module (to charge the card), the inventory module (to reduce stock), and the notification module (to send a confirmation email). Integration testing verifies that these modules interact correctly: Does the order module correctly pass payment information to the payment module? Does a successful payment correctly trigger an inventory update? Does a failed payment correctly prevent the order from being finalized?

Approaches to integration testing:

- **Big Bang:** All modules are integrated at once and tested together. Simple but hard to isolate the source of failures.
- **Top-Down:** Testing starts with the top-level modules and works downward, using stubs (simplified stand-ins) for lower-level modules not yet integrated.
- **Bottom-Up:** Testing starts with the lowest-level modules and works upward, using drivers (simplified callers) to invoke the lower-level modules.
- **Sandwich / Hybrid:** Combines top-down and bottom-up approaches.

System Testing

Definition

System Testing tests the complete, fully integrated software system as a whole to verify that it meets all specified requirements. It is performed on the entire application in an environment that closely mimics the production environment.

Who does it: QA/Testing team (typically a dedicated testing team, not developers).

When it happens: After integration testing, when the complete system is assembled.

Real-world example: Testing the entire Netflix application end-to-end: a user can sign up, log in, browse the catalog, search for a title, play a video, adjust playback settings (subtitles, audio language, quality), add a title to "My List," manage their subscription, and log out. System testing verifies that all of these features work together as a complete, coherent system.

What it covers: System testing evaluates both functional requirements (does each feature work?) and non-functional requirements (is it fast enough? secure enough? usable enough?).

User Acceptance Testing (UAT)

Definition

User Acceptance Testing (UAT) is the final level of testing, performed by end users or client representatives to determine whether the software meets their business needs and is ready for production release. It validates the system against real-world business scenarios, not just technical specifications.

Who does it: End users, business stakeholders, or client representatives — not the development or QA team.

When it happens: After system testing, as the last testing gate before production release.

Why it exists: Technical testers test against specifications, but specifications can be incomplete or can fail to capture the nuances of how real users actually work. UAT bridges this gap by having the people who will actually use the software verify that it fits their workflow.

Real-world example: A bank develops a new loan processing system. After the QA team performs system testing, actual loan officers (the end users) perform UAT by processing sample loan applications through the system using their real workflows. They check whether the system supports their actual business process, whether the interfaces make sense for their daily work, and whether the reports meet regulatory requirements.

Types of UAT:

- **Alpha Testing:** UAT performed by users at the developer's site, under the developer's guidance and observation.
- **Beta Testing:** UAT performed by users at their own site, in their own environment, without the developer present. Beta testing is exposed to a broader set of real-world conditions.

Smoke Testing

Definition

Smoke Testing (also called **Build Verification Testing**) is a quick, shallow, high-level test executed on a new software build to verify that the most critical functionalities work. It determines whether the build is stable enough for further, more detailed testing.

Analogy: The term comes from hardware testing — when you first power on a new electronic circuit board, you check whether smoke comes out. If it does, you know something is fundamentally wrong and there is no point testing individual components. If no smoke appears, the board is stable enough for detailed testing.

Who does it: QA team, or automated as part of the CI/CD pipeline.

When it happens: Immediately after a new build is deployed to the test environment, *before* any other testing begins.

What it covers: Only the critical, core functionalities. For an e-commerce site, a smoke test might cover: Can the homepage load? Can a user log in? Can a user search for a product? Can a user add an item to the cart? Can the checkout page load? If any of these fail, the build is rejected — there is no point testing minor features if the major ones are broken.

Real-world example: At a ride-sharing company, the smoke test suite for each new build might verify: (1) The app launches without crashing, (2) A user can log in, (3) The map displays correctly, (4) A ride can be requested, (5) A driver can accept a ride. If any of these five tests fail, the build is rejected and sent back to development.

Sanity Testing

Definition

Sanity Testing is a focused, narrow test performed after receiving a software build with minor changes (bug fixes or small feature additions) to verify that the specific changes work correctly and that the related areas have not been broken.

Key difference from Smoke Testing: Smoke testing is broad and shallow (tests many core features quickly). Sanity testing is narrow and deep (tests specific changed areas thoroughly).

Aspect	Smoke Testing	Sanity Testing
Breadth	Wide — covers all critical features	Narrow — focuses on specific changes
Depth	Shallow — quick check of each feature	Deep — thorough check of changed area
When	Every new build, before any testing	After a specific bug fix or small change
Purpose	Is the build stable enough to test?	Does this specific fix/change work correctly?
If it fails	Entire build is rejected	Only the specific change is sent back
Analogy	Checking that all rooms in a house have electricity	Checking that a specific repaired light switch now works

Real-world example: A developer fixes a bug where the "Apply Coupon" button was not working on the checkout page. Sanity testing would focus specifically on: (1) Does the coupon button now work with a valid coupon? (2) Does it correctly reject an expired coupon? (3) Does the discount calculate correctly? (4) Does the total price update properly after the coupon is applied? (5) Does the coupon still work after the user modifies their cart?

Regression Testing

Definition

Regression Testing is re-running previously passing test cases after code changes (bug fixes, new features, configuration changes, or upgrades) to ensure that the changes have not introduced new defects or broken existing functionality.

Why it exists: In complex software systems, components are interconnected. Fixing a bug in one module can inadvertently break a feature in a completely different module. Regression testing catches these unintended side effects.

Analogy: Imagine a building where fixing a water leak on the third floor accidentally disrupts the plumbing on the fifth floor. Regression testing is like re-checking every floor's plumbing after any repair, not just the floor where the repair was made.

Why it is the #1 candidate for automation: Regression tests must be run frequently (ideally with every code change), they cover a large and ever-growing number of test cases, they are repetitive (the same tests run over and over), and they are time-consuming to run manually. This makes regression testing the perfect candidate for test automation — and this is exactly what you will learn to do with Selenium later in this workshop.

Real-world example: Instagram adds a new "Reels" feature. After the new feature is deployed, the regression test suite re-runs all existing tests for Stories, Feed, Direct Messages, Profile, Search, and Notifications to ensure that the addition of Reels did not break any of these existing features. At Instagram's scale, this regression suite contains tens of thousands of automated tests that run in minutes via CI/CD.

3.3 Non-Functional Testing Types

Non-functional testing evaluates *how well* the system performs, rather than *what* it does. It focuses on quality attributes like speed, security, usability, and reliability.

Performance Testing

Definition

Performance Testing evaluates the speed, responsiveness, stability, and resource usage of a software application under various conditions.

Performance testing has several sub-types:

Sub-type	What It Tests	Real-World Scenario
Load Testing	System behavior under expected (normal) load	Can Netflix handle 10 million users streaming simultaneously during normal evening hours?
Stress Testing	System behavior beyond its expected maximum capacity, to find the breaking point	What happens to Netflix when 50 million users suddenly try to stream during the premiere of a major show? At what point does the system degrade or crash?
Spike Testing	System behavior when load suddenly spikes and then drops	Can Amazon handle the sudden traffic surge when a flash sale begins and ends?

Sub-type	What It Tests	Real-World Scenario
Endurance (Soak) Testing	System stability under sustained load over a long period	Does the banking app remain stable after 72 hours of continuous operation, or do memory leaks cause it to slow down over time?
Scalability Testing	How well the system scales (adds capacity) to handle increasing load	If we add 10 more servers, does throughput increase proportionally?

You will use Apache JMeter for performance testing in Hours 9–11 of this workshop.

Security Testing

Security testing identifies vulnerabilities, threats, and risks in the software that could lead to unauthorized access, data breaches, or other security compromises.

Common security concerns tested:

- **Authentication:** Can only authorized users log in? Can someone bypass the login?
- **Authorization:** Can a regular user access admin-only features?
- **Data protection:** Is sensitive data (passwords, credit card numbers) encrypted?
- **Input validation:** Can an attacker inject malicious code through input fields (SQL injection, Cross-Site Scripting / XSS)?
- **Session management:** Does the session time out after inactivity? Can a session token be hijacked?

Real-world example: A security tester at a banking app might attempt to access another user's account details by modifying the account ID in the URL (e.g., changing /account/12345 to /account/12346). If the system shows the other user's information, it has an **Insecure Direct Object Reference (IDOR)** vulnerability — a critical security bug.

Usability Testing

Usability testing evaluates how easy and intuitive the software is for end users. It focuses on the user experience: Can users accomplish their tasks efficiently and without confusion?

Real-world example: A usability test for a food delivery app might involve watching real users try to order food for the first time. Testers observe: Can users find the "search" function? Do they understand how to filter restaurants? Is the checkout process confusing?

Do they know how to apply a promo code? If multiple users struggle with the same step, it signals a usability problem.

Compatibility Testing

Compatibility testing verifies that the software works correctly across different environments — browsers, operating systems, devices, screen sizes, and network conditions.

Dimensions of compatibility:

- **Browser compatibility:** Chrome, Firefox, Safari, Edge
- **Operating system compatibility:** Windows, macOS, Linux, iOS, Android
- **Device compatibility:** Desktop, tablet, mobile (various screen sizes and resolutions)
- **Network compatibility:** Wi-Fi, 4G, 3G, slow connections
- **Backward compatibility:** Does the new version work with data or configurations created by the old version?

Real-world example: A web application tested and working perfectly on Chrome on a Windows desktop might display a completely broken layout on Safari on an iPhone. Compatibility testing catches these discrepancies before users encounter them.

3.4 Testing Based on Knowledge of Code

This categorization is based on how much the tester knows about the internal structure (source code) of the software being tested.

Black-Box Testing

Definition

Black-Box Testing treats the software as a "black box" — the tester has no knowledge of or access to the internal code, architecture, or implementation. Tests are based entirely on the external specifications (requirements, user stories). The tester provides inputs and observes outputs without knowing *how* the software produces those outputs.

Who does it: QA testers, end users (during UAT), business analysts.

Focus: What the software does, not how it does it.

Techniques: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing (all covered in detail in Hour 4).

Real-world example: Testing a calculator app's addition function: you enter "2 + 3" and expect "5." You do not look at the source code to see how the addition is implemented — you only care whether the output is correct.

White-Box Testing

Definition

White-Box Testing (also called Clear-Box, Glass-Box, or Structural Testing) tests the internal structure, logic, and code of the software. The tester has full access to and knowledge of the source code and designs tests based on code paths, branches, conditions, and loops.

Who does it: Developers (primarily), or SDETs with strong programming skills.

Focus: How the software does what it does — internal paths, code coverage, logic flow.

Techniques: Statement coverage, branch coverage, path coverage, condition coverage.

Real-world example: A developer writing unit tests for a function that calculates insurance premiums would examine the code to ensure every if/else branch is tested: the path for young drivers, old drivers, drivers with accidents, drivers without accidents, etc.

Grey-Box Testing

Definition

Grey-Box Testing combines elements of both black-box and white-box testing. The tester has partial knowledge of the internal workings (e.g., access to the database schema, knowledge of the architecture, understanding of the API contracts) but tests the system primarily through its external interfaces.

Who does it: QA engineers with some technical knowledge, especially those doing API testing or database testing.

Real-world example: An API tester testing a REST API endpoint knows the expected request format, response format, and database structure, but does not read the source code of the endpoint implementation. They use their knowledge of the database to verify that the API call correctly created or updated a database record. This is grey-box testing — more informed than pure black-box but not fully white-box.

Comparison Table

Aspect	Black-Box	White-Box	Grey-Box
Code knowledge	None	Full access to source code	Partial (architecture, DB, APIs)
Tester profile	QA tester, end user	Developer, SDET	Technical QA engineer
Focus	What the software does (behavior)	How the software does it (structure)	Both behavior and partial structure
Basis for tests	Requirements, specifications	Source code, design documents	Requirements + partial technical knowledge
Test level	System testing, UAT	Unit testing, integration testing	Integration testing, API testing
Techniques	EP, BVA, Decision Tables	Statement/branch/path coverage	API testing, database verification

3.5 Testing Based on Approach

Manual Testing

Definition

Manual Testing is the process of a human tester executing test cases by interacting with the software directly — clicking buttons, entering data, navigating through screens — without using any automation tools or scripts.

When to use manual testing:

- Exploratory testing (discovering unexpected behaviors through creative exploration)
- Usability testing (evaluating how intuitive the interface is — requires human judgment)
- Ad-hoc testing (unscripted testing based on the tester's experience and intuition)
- Testing new features for the first time (before stable enough for automation)
- Tests that are run rarely (one-time tests, setup verification)

Advantages: Flexible, can discover unexpected issues, requires no programming skill, best for evaluating user experience and visual design, good for exploratory and ad-hoc testing.

Limitations: Slow, error-prone (humans make mistakes during repetitive tasks), not scalable, expensive for regression testing (the same tests must be re-executed manually every time).

Automated Testing

Definition

Automated Testing uses specialized software tools and scripts to execute test cases automatically, compare actual results to expected results, and report pass/fail status — all without human intervention during execution.

When to use automated testing:

- Regression testing (the top use case — repetitive tests that run with every build)
- Performance and load testing (impossible to simulate thousands of users manually)
- Tests that must run on multiple configurations (many browsers, many devices)
- Smoke testing (run automatically on every new build in CI/CD)
- Tests with large data sets or many input combinations
- Tests that run frequently (daily, hourly, or with every code commit)

Advantages: Fast, reliable (no human errors during execution), reusable, scalable, enables CI/CD, provides consistent results, can run 24/7 unattended.

Limitations: High initial investment (time to write and maintain scripts), cannot evaluate subjective qualities (usability, visual appeal), requires programming skills, brittle if the UI changes frequently (scripts break), not suitable for exploratory testing.

In this workshop, you will learn automated testing with three industry-standard tools:

- **Postman** — for API testing automation (Hours 6–8)
- **Apache JMeter** — for performance testing automation (Hours 9–11)
- **Selenium with Java** — for web UI testing automation (Hours 12–15)

Aspect	Manual Testing	Automated Testing
Execution	Human tester	Script/tool
Speed	Slow	Fast (often seconds for thousands of checks)

Aspect	Manual Testing	Automated Testing
Reliability	Prone to human error	Consistent and repeatable
Initial cost	Low (just human time)	High (framework setup, script development)
Maintenance cost	Low	Moderate to high (scripts must be updated when the application changes)
Best for	Exploratory testing, usability, ad-hoc, new features	Regression, performance, smoke, CI/CD, multi-config
Scalability	Very limited	Highly scalable
Programming needed	No	Yes

3.6 Master Comparison Table

This table compares the most commonly confused testing types — a frequent source of interview questions.

Testing Type	Purpose	Scope	Depth	When	Who
Smoke	Is the build stable?	All critical features	Shallow	Every new build	QA / CI pipeline
Sanity	Does this specific fix work?	Only changed area	Deep	After specific fix	QA
Regression	Did changes break existing features?	All previously tested features	Deep	After every change	QA / Automation
Unit	Does this one function work?	One function/method	Very deep	During development	Developers

Testing Type	Purpose	Scope	Depth	When	Who
Integration	Do modules work together?	Interfaces between modules	Deep	After unit testing	Developers / QA
System	Does the whole system work?	Entire application	Broad and deep	After integration testing	QA team
UAT	Does it meet business needs?	Business scenarios	Varies	Before production release	End users / clients

3.7 Hour 3 Interview Corner

Q1: What is the difference between Smoke Testing and Sanity Testing? Model Answer:

Smoke Testing is a broad, shallow test that checks all critical functionalities of a new build to determine if it is stable enough for further testing. It answers the question "Is the build stable?" Sanity Testing is a narrow, deep test that verifies a specific bug fix or small change works correctly. It answers the question "Does this particular fix work?" If smoke testing fails, the entire build is rejected. If sanity testing fails, only the specific change is sent back for rework.

Q2: What is Regression Testing? Why is it important? Model Answer: Regression Testing is the re-execution of previously passing test cases after code changes to ensure that modifications have not introduced new defects or broken existing functionality. It is critically important because in complex software, components are interconnected and a change in one area can have unintended effects in other areas. Regression testing is the primary candidate for test automation because of its repetitive, high-frequency nature.

Q3: Explain the difference between Black-Box and White-Box testing. Model Answer:

Black-Box Testing treats the software as an opaque box — the tester tests based solely on requirements and specifications without any knowledge of the internal code. White-Box Testing requires full access to the source code and tests the internal structure, logic paths, and code branches. Black-box testing is typically done by QA testers during system and acceptance testing, while white-box testing is typically done by developers during unit testing.

Q4: What types of testing cannot be automated? Model Answer: Exploratory testing, usability testing, ad-hoc testing, and any testing that requires subjective human judgment

(visual assessment, user experience evaluation) cannot be effectively automated. Automated tests can only check what they are programmed to check — they cannot "notice" unexpected issues the way a human tester can during exploratory testing.

Q5: When would you choose manual testing over automated testing? Model Answer: I would choose manual testing for exploratory testing (discovering unknown issues through creative exploration), usability testing (evaluating user experience requires human judgment), testing new or frequently changing features (automation scripts would break constantly), one-time tests (the automation investment would not be justified), and ad-hoc testing based on intuition and experience.

Hour 4 — Test Case Design Techniques

4.1 What Are Test Case Design Techniques?

In Hour 1, we learned that exhaustive testing is impossible — we cannot test every possible input combination. So how do we decide *which* test cases to write? We need a systematic, structured approach that helps us select a small number of test cases that give us maximum coverage and maximum confidence.

Definition

Test Case Design Techniques (also called Test Design Techniques or Test Specification Techniques) are systematic methods used to derive a set of test cases from requirements and specifications. They help testers select test inputs that are most likely to reveal defects while keeping the total number of test cases manageable.

Why they matter: Without these techniques, testers tend to test randomly or based on gut feeling, which leads to inefficient testing with low coverage. Formal techniques ensure that testing is systematic, repeatable, and thorough. They are also a key part of the ISTQB curriculum and a very common interview topic.

Categories:

Category	Description	Techniques
Black-Box Techniques	Based on requirements/specifications, no code knowledge needed	Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing

Category	Description	Techniques
Experience-Based Techniques	Based on the tester's knowledge and intuition	Error Guessing, Exploratory Testing
White-Box Techniques	Based on source code structure	Statement Coverage, Branch Coverage, Path Coverage

This hour focuses on the black-box and experience-based techniques, as these are the techniques you will use most frequently as a QA tester.

4.2 Equivalence Partitioning (EP)

The Concept

Definition

Equivalence Partitioning (EP) is a test design technique that divides the input domain of a software function into groups (called **equivalence classes** or **partitions**) such that all values within a single partition are expected to be treated identically by the software. One representative value from each partition is selected for testing, based on the assumption that if one value in the partition works correctly, all other values in that partition will also work correctly.

The core idea in simple terms: If a function accepts ages 18–60, then testing with age 25 should give us the same result as testing with age 30, 35, or 40 — because they are all in the same "valid" group. So instead of testing all 43 valid ages, we test just one from the valid group, one from the "too young" group (below 18), and one from the "too old" group (above 60).

Why It Exists

Without EP, a tester might randomly pick a few values to test, potentially missing entire categories of input. EP ensures that every meaningful category of input is represented in the test suite, while dramatically reducing the number of tests needed.

Step-by-Step Process

1. Identify the input conditions from the requirements

2. Divide each input condition into valid equivalence classes (inputs that should be accepted) and invalid equivalence classes (inputs that should be rejected or handled as errors)
3. Select one representative value from each equivalence class
4. Create one test case for each representative value

Worked Example

Requirement: A registration form has a "User Age" field that accepts ages from 18 to 60 (inclusive). Users below 18 or above 60 should see an error message.

Step 1 — Identify input condition: Age field, valid range 18–60.

Step 2 — Define equivalence classes:

Partition ID	Partition	Value Range	Type	Expected Behavior
EP1	Below minimum age	Less than 18 (e.g., 0–17)	Invalid	Error: "You must be at least 18 years old"
EP2	Valid age range	18–60	Valid	Registration proceeds
EP3	Above maximum age	Greater than 60 (e.g., 61+)	Invalid	Error: "Maximum age is 60"

Step 3 — Select representative values:

Partition	Representative Value	Test Case
EP1 (below 18)	10	Enter age 10, expect error message
EP2 (18–60)	35	Enter age 35, expect success
EP3 (above 60)	75	Enter age 75, expect error message

Result: Instead of testing all possible ages, we test just 3 values — one from each partition. If the software handles 35 correctly, we have reasonable confidence it handles all ages 18–60 correctly.

Additional Partitions to Consider

Experienced testers also consider these additional equivalence classes:

Partition	Representative Value	Expected Behavior
Non-numeric input	"abc"	Error: "Please enter a valid number"
Negative number	-5	Error
Zero	0	Error
Decimal number	25.5	Depends on requirement (likely error if only integers are accepted)
Empty input	(blank)	Error: "Age is required"
Very large number	999999	Error

Industry Best Practice

Always consider at least one invalid partition for each type of "wrong" input, not just one invalid partition overall. The software may handle "age below 18" correctly but crash when given "abc" as input. Each type of invalid input exercises different validation logic and may reveal different defects.

Common Beginner Mistake

Beginners often create equivalence classes based on output behavior rather than input characteristics. The correct approach is to partition the *input domain* — what goes into the system — not the output. The output is the expected behavior associated with each input partition.

4.3 Boundary Value Analysis (BVA)

The Concept

Definition

Boundary Value Analysis (BVA) is a test design technique that focuses on testing values at the exact boundaries (edges) of equivalence partitions. It is based on the observation that

defects tend to cluster at the boundaries between partitions — at the exact point where behavior changes from "valid" to "invalid."

The core idea in simple terms: If a field accepts ages 18–60, bugs are most likely to occur at exactly 17, 18, 60, and 61 — the boundary values — rather than at random values in the middle like 35.

Why Boundaries Are Bug-Prone

Boundary bugs are extremely common because developers frequently make **off-by-one errors** in their code. Consider a developer who writes: if (age > 18) instead of if (age >= 18). This bug would incorrectly reject age 18 (which should be valid). Testing with age 35 would never catch this bug because 35 passes both conditions. Only testing with the boundary value 18 would reveal it.

Step-by-Step Process

For each boundary between equivalence partitions, test these values:

1. The value just below the boundary (one less than the lower boundary)
2. The exact lower boundary value
3. The value just above the lower boundary (one more than the lower boundary)
4. The value just below the upper boundary (one less than the upper boundary)
5. The exact upper boundary value
6. The value just above the upper boundary (one more than the upper boundary)

Worked Example

Requirement: Same as before — "User Age" field accepts ages 18 to 60 (inclusive).

Boundary values to test:

Test Value	Relation to Boundary	Expected Result	What It Catches
17	Just below lower boundary	Invalid (error)	Off-by-one at lower bound (e.g., > 18 instead of >= 18)
18	Exact lower boundary	Valid (accepted)	Incorrect lower bound check

Test Value	Relation to Boundary	Expected Result	What It Catches
19	Just above lower boundary	Valid (accepted)	Off-by-one at lower bound
59	Just below upper boundary	Valid (accepted)	Off-by-one at upper bound
60	Exact upper boundary	Valid (accepted)	Incorrect upper bound check
61	Just above upper boundary	Invalid (error)	Off-by-one at upper bound (e.g., < 60 instead of <= 60)

Result: 6 test cases from BVA, which can be combined with the 3 from EP to create a comprehensive set of 9 test cases that efficiently cover the age input.

EP + BVA Combined

In practice, EP and BVA are always used together. EP ensures every partition is covered; BVA ensures the boundaries are tested precisely.

graph LR

A["Invalid Low
EP: age=10
BVA: age=17"] --- B["Boundary
18"]

B --- C["Valid Range
EP: age=35
BVA: 18, 19, 59, 60"]

C --- D["Boundary
60"]

D --- E["Invalid High
EP: age=75
BVA: age=61"]

style A fill:#ef4444,stroke:#333,color:#fff

style B fill:#f97316,stroke:#333,color:#fff

style C fill:#22c55e,stroke:#333,color:#fff

style D fill:#f97316,stroke:#333,color:#fff

style E fill:#ef4444,stroke:#333,color:#fff

Common Beginner Mistake

Beginners often skip BVA, thinking that EP alone is sufficient. However, EP tests only representative values from the middle of each partition — it does not test the precise boundaries where most bugs actually hide. Always apply both EP and BVA together for thorough coverage.

4.4 Decision Table Testing

The Concept

Definition

Decision Table Testing is a test design technique used when software behavior depends on combinations of multiple input conditions. It creates a table that systematically lists all possible combinations of conditions and the expected action (output) for each combination, ensuring no combination is missed.

When to use it: When the software has business rules that depend on multiple conditions evaluated together. "If condition A is true AND condition B is false, then do X; if condition A is false AND condition B is true, then do Y," etc.

The core idea in simple terms: If a decision depends on two yes/no conditions, there are $2 \times 2 = 4$ possible combinations. A decision table lists all four and specifies what should happen in each case, ensuring you test every combination.

Worked Example

Requirement: An online store offers free shipping based on these rules:

- Customers who have a premium membership get free shipping
- Orders above \$100 get free shipping
- All other orders are charged \$10 for shipping

Step 1 — Identify the conditions:

- Condition 1: Is the customer a premium member? (Yes / No)
- Condition 2: Is the order amount above \$100? (Yes / No)

Step 2 — Identify the possible actions:

- Action 1: Free shipping
- Action 2: Charge \$10 shipping

Step 3 — Build the decision table:

Rule	Premium Member?	Order > \$100?	Expected Action
Rule 1	Yes	Yes	Free shipping
Rule 2	Yes	No	Free shipping (premium membership overrides)
Rule 3	No	Yes	Free shipping (order amount qualifies)
Rule 4	No	No	Charge \$10 shipping

Step 4 — Create one test case for each rule:

Test Case Premium Member Order Amount Expected Shipping

TC1	Yes	\$150	Free
TC2	Yes	\$50	Free
TC3	No	\$150	Free
TC4	No	\$50	\$10

Result: 4 test cases that cover every possible combination of conditions. Without the decision table, a tester might test only Rules 1 and 4 (the "obvious" combinations) and miss Rules 2 and 3.

Scalability Note

With n binary (yes/no) conditions, there are 2^n possible combinations. Two conditions = 4 rules. Three conditions = 8 rules. Four conditions = 16 rules. For large numbers of conditions, techniques like "collapsed decision tables" (combining rules with identical actions) are used to keep the number manageable.

4.5 State Transition Testing

The Concept

Definition

State Transition Testing is a test design technique used when the software's behavior changes based on its current state and the event that triggers a transition. It models the system as a finite state machine — a set of states, transitions between states, events that trigger transitions, and actions that occur during transitions.

When to use it: When the software has defined states and the behavior for the same event depends on which state the system is currently in. Classic examples include login attempts, order processing, ATM transactions, and subscription management.

The core idea in simple terms: An ATM card can be in different states: "Active," "Blocked," or "Expired." What happens when you try to use the card depends on which state it is currently in. If the card is Active, your transaction proceeds. If the card is Blocked, the transaction is rejected. State Transition Testing systematically tests all relevant state changes.

Worked Example

Requirement: A user account login system with the following rules:

- A user starts in the "Locked Out" state after 3 consecutive failed login attempts
- A successful login at any point resets the failure counter and grants access
- After being locked out, the user must wait 30 minutes or reset their password

State Transition Diagram:

stateDiagram-v2

[*] --> S0: Account Created

S0 --> S1: Failed Login (attempt 1)

S0 --> LoggedIn: Successful Login

S1 --> S2: Failed Login (attempt 2)

S1 --> LoggedIn: Successful Login

S2 --> LockedOut: Failed Login (attempt 3)

S2 --> LoggedIn: Successful Login

LockedOut --> S0: Wait 30 min / Reset Password

LoggedIn --> S0: Logout

S0: No Failed Attempts

S1: 1 Failed Attempt

S2: 2 Failed Attempts

State Transition Table:

Current State Event		Next State	Action
S0 (no failures)	Valid login	LoggedIn	Grant access, reset counter
S0 (no failures)	Invalid login	S1 (1 failure)	Show error, increment counter
S1 (1 failure)	Valid login	LoggedIn	Grant access, reset counter
S1 (1 failure)	Invalid login	S2 (2 failures)	Show error, increment counter
S2 (2 failures)	Valid login	LoggedIn	Grant access, reset counter
S2 (2 failures)	Invalid login	LockedOut	Lock account, show lockout message
LockedOut	Wait 30 min / Reset password	S0	Unlock account, reset counter

Test cases derived from the table:

Test Case Sequence of Events		Expected Final State
TC1	Correct password on first attempt	LoggedIn
TC2	Wrong → Correct	LoggedIn
TC3	Wrong → Wrong → Correct	LoggedIn
TC4	Wrong → Wrong → Wrong	LockedOut
TC5	Wrong → Wrong → Wrong → Wait 30 min → Correct	LoggedIn

This technique ensures that every state and every transition is tested, catching bugs that other techniques might miss.

4.6 Error Guessing

The Concept

Definition

Error Guessing is an experience-based test design technique where the tester uses their knowledge, intuition, and experience with similar applications to anticipate where defects are likely to occur, and designs test cases specifically targeting those areas.

Unlike the formal techniques above, error guessing is not systematic — it relies on the tester's "gut feeling" developed through experience. It is most effective when used *in addition to* formal techniques, not as a replacement.

Common Error Guessing Targets

Category	What to Test	Examples
Null/Empty inputs	What happens when a required field is left blank?	Empty username, empty password, empty search query
Zero values	What happens with zero quantity, zero price, zero balance?	Add 0 items to cart, transfer \$0
Negative values	What happens with negative numbers where only positive are expected?	Quantity: -1, Age: -5
Special characters	What happens with characters like < > ' ' & / \ ; -- #?	Name: O'Brien, Search: <script>alert('XSS')</script>
Extremely large inputs	What happens when input exceeds expected size?	Paste 10,000 characters into a name field
Duplicate submissions	What happens when the user submits a form twice?	Double-clicking the "Pay Now" button
Concurrent access	What happens when two users modify the same data simultaneously?	Two agents booking the last seat on a flight

Category	What to Test	Examples
Network interruption	What happens if the connection drops mid-operation?	Wi-Fi disconnects during a file upload
Date/Time edge cases	Leap years, daylight saving time, time zones, year boundaries	Feb 29, Dec 31 to Jan 1 transition

Industry Best Practice

Experienced testers maintain personal "error guessing checklists" — lists of common bug patterns they have encountered over their careers. These checklists grow over time and become increasingly valuable. Many companies also maintain organizational knowledge bases of known bug patterns for their specific products.

4.7 Hour 4 Interview Corner

Q1: What is Equivalence Partitioning? Give an example. Model Answer: Equivalence Partitioning is a black-box test design technique that divides the input domain into groups (partitions) where all values within a partition are expected to be treated identically by the software. One representative value from each partition is tested, reducing the number of test cases while maintaining coverage. For example, for a field accepting ages 18–60, I would create three partitions: below 18 (invalid, test with 10), 18–60 (valid, test with 35), and above 60 (invalid, test with 75).

Q2: What is Boundary Value Analysis? Why is it used with EP? Model Answer: Boundary Value Analysis is a technique that tests values at the exact edges of equivalence partitions — the boundary values — because defects disproportionately cluster at boundaries due to common programming errors like off-by-one mistakes. It is used with EP because EP tests representative values from the *middle* of partitions, while BVA specifically tests the *edges*. Together, they provide both breadth (every partition) and precision (every boundary).

Q3: When would you use Decision Table Testing? Model Answer: Decision Table Testing is used when the system's behavior depends on combinations of multiple input conditions. For example, if shipping cost depends on both membership status and order amount, a decision table ensures every combination of those conditions is tested. It is especially valuable for business rule validation where missing a single condition combination could lead to a costly business logic bug.

Q4: What is State Transition Testing? Give an example. Model Answer: State Transition Testing models the system as having defined states with transitions triggered by events. It is used when the same event can produce different outcomes depending on the system's current state. A classic example is a login system where 3 consecutive failed login attempts lock the account. State Transition Testing would test all paths: successful login on first attempt, successful login after 1 failure, successful login after 2 failures, and lockout after 3 failures.

Hour 5 — Writing Test Cases, Documentation & Bug Reporting

5.1 Anatomy of a Test Case

What Is a Test Case?

Definition

A **Test Case** is a documented set of conditions, inputs, actions, and expected results that a tester follows to verify whether a specific feature or function of the software works correctly. It is the fundamental unit of testing — every test that is executed, whether manually or through automation, starts as a test case.

Think of a test case as a "recipe" for a specific test. Just as a cooking recipe has ingredients (inputs), steps (actions), and an expected result (the finished dish), a test case has test data, execution steps, and expected outcomes.

The Standard Test Case Template

Every professional test case includes these fields:

Field	Description	Example
Test Case ID	A unique identifier for the test case, following a consistent naming convention	TC_LOGIN_001
Test Case Title	A brief, descriptive name summarizing what the test case verifies	Verify successful login with valid email and password
Module/Feature	The feature or module being tested	Login / Authentication

Field	Description	Example
Priority	How important it is to execute this test case (High, Medium, Low)	High
Preconditions	Conditions that must be true before the test can be executed	User account exists with email "user@test.com" and password "ValidPass123"
Test Data	The specific data values to be used during the test	Email: user@test.com, Password: ValidPass123
Test Steps	A numbered, step-by-step sequence of actions the tester must perform	1. Navigate to login page. 2. Enter email. 3. Enter password. 4. Click "Login" button.
Expected Result	The precise outcome that should occur if the software is working correctly	User is redirected to the dashboard. A welcome message "Hello, User" is displayed.
Actual Result	The outcome that actually occurred during test execution (filled in during execution)	<i>(Filled during execution)</i>
Status	The test result: Pass, Fail, Blocked, or Not Executed	<i>(Updated after execution)</i>
Postconditions	The state of the system after the test is executed	User is logged in and session is active
Comments/Notes	Any additional information, observations, or context	Tested on Chrome v120, Windows 11

Complete Test Case Example

Test Case ID: TC_LOGIN_001

Title: Verify successful login with valid credentials

Module: Authentication / Login

Priority: High

Preconditions:

1. The application is accessible at <https://app.example.com>
2. A registered user account exists with email "user@test.com" and password "ValidPass123"
3. The user is not currently logged in

Test Data:

- Email: user@test.com
- Password: ValidPass123

Test Steps:

Step #	Action	Expected Result
1	Open the browser and navigate to https://app.example.com/login	Login page is displayed with Email and Password fields and a "Login" button
2	Enter "user@test.com" in the Email field	Email is displayed in the field
3	Enter "ValidPass123" in the Password field	Password is masked (displayed as dots or asterisks)
4	Click the "Login" button	User is redirected to the Dashboard page. The URL changes to /dashboard. A welcome message "Hello, User" is displayed in the top-right corner.

Postconditions: User is logged in. Session is active. User can access all authenticated pages.

5.2 Characteristics of a Good Test Case

Not all test cases are created equal. A well-written test case is the difference between a test suite that reliably catches bugs and one that wastes time and creates confusion.

The Eight Hallmarks of Quality Test Cases

1. Atomic (One Thing Per Test Case) Each test case should verify exactly one behavior or scenario. Do not combine "verify login with valid credentials" and "verify error message for invalid password" in a single test case. If a combined test case fails, you cannot immediately tell *which* behavior failed.

2. Clear and Unambiguous Another tester — one who has never seen this test case before — should be able to execute it without any questions. Steps should be precise: "Click the blue 'Submit' button at the bottom of the form" is better than "Submit the form."

3. Traceable to a Requirement Every test case should trace back to a specific requirement or user story. If it does not, it either tests something unnecessary or there is a missing requirement. This traceability is captured in the Requirement Traceability Matrix (RTM).

4. Repeatable Executing the same test case multiple times should produce the same result every time (assuming the software has not changed). If a test case depends on a specific state that changes after execution (e.g., "register a new account with email X" — the email is now taken for future runs), the test must include setup/teardown steps or use dynamically generated test data.

5. Self-Contained A test case should include all the information needed to execute it — preconditions, test data, steps, and expected results. The tester should not need to look up information elsewhere.

6. Maintainable When the application changes, test cases must be updated. Well-structured test cases with clear separation of test data from test logic are easier to maintain. Using reusable precondition blocks and shared test data sets also reduces maintenance effort.

7. Appropriately Prioritized Not all test cases are equally important. A test case for the "complete a purchase" flow is more critical than one for "change profile picture." Prioritization ensures that the most important tests are executed first, especially when time is limited.

8. Has Precise Expected Results Vague expected results like "page loads correctly" or "system works fine" are not acceptable. Expected results must be specific and verifiable: "The page loads within 3 seconds," "The user's name 'John Smith' is displayed in the header," "The order total is \$54.99 including 10% tax."

Common Beginner Mistake

The most common mistake beginners make is writing vague expected results. "System should work properly" is not a testable expected result. Always ask yourself: "If I showed

this expected result to a stranger, could they objectively determine whether the test passed or failed?" If the answer is no, make it more specific.

5.3 Test Case Documentation Best Practices

Naming Conventions

Use a consistent, meaningful naming convention for test case IDs:

Convention	Example	Meaning
TC_[Module]_[Number]	TC_LOGIN_001	Test case #1 for the Login module
TC_[Module]_[Scenario]_[Number]	TC_LOGIN_VALID_001	Test case #1 for valid login scenario
[Project]_TC_[Module]_[Number]	ECOM_TC_CART_005	E-commerce project, Cart module, test #5

Organizing Test Cases

For any real project, you will have hundreds or thousands of test cases. Organization is critical.

Group test cases by:

- Module or Feature (Login, Registration, Cart, Checkout, Search, Profile)
- Test Type (Functional, Regression, Smoke, Sanity)
- Priority (Critical, High, Medium, Low)

Use test management tools: In professional environments, test cases are stored in dedicated test management tools such as TestRail, Zephyr (a Jira plugin), Azure Test Plans, qTest, or (for smaller teams) well-structured spreadsheets (Google Sheets or Excel).

Requirement Traceability Matrix (RTM)

Definition

A **Requirement Traceability Matrix (RTM)** is a document that maps every requirement to the test cases that verify it, and vice versa. It ensures complete test coverage (every requirement has at least one test case) and helps identify gaps (requirements without test cases) and redundancies (test cases that do not map to any requirement).

Requirement ID	Requirement Description	Test Case IDs	Status
REQ_001	User can log in with valid email and password	TC_LOGIN_001, TC_LOGIN_002	Covered
REQ_002	User sees error for invalid password	TC_LOGIN_003, TC_LOGIN_004	Covered
REQ_003	Account locks after 3 failed attempts	TC_LOGIN_010, TC_LOGIN_011	Covered
REQ_004	User can reset password via email	(none)	GAP — needs test cases

The RTM is a living document that is updated throughout the project. It is one of the primary tools for measuring test coverage and is a key deliverable in the STLC.

5.4 Bug Reporting

Finding a bug is only half the battle. The other half — and arguably the more important half — is communicating that bug clearly enough that a developer can understand it, reproduce it, and fix it. A poorly written bug report wastes everyone's time: the developer cannot reproduce it, asks for more information, the tester has to re-investigate, and the bug sits unfixed while this back-and-forth happens.

What Is a Bug Report?

Definition

A **Bug Report** (also called a Defect Report) is a formal document that describes a defect found during testing. It provides all the information a developer needs to understand, reproduce, and fix the defect.

The Standard Bug Report Template

Field	Description	Example
Bug ID	Unique identifier, usually auto-	BUG-2847

Field	Description	Example
	generated by the bug tracking tool	
Title	A concise, descriptive summary of the bug (should be searchable)	"Apply Coupon" button returns error 500 for valid percentage coupons on checkout page
Severity	How badly the bug impacts the system (Critical, Major, Minor, Trivial)	Major
Priority	How urgently the bug needs to be fixed (High, Medium, Low)	High
Reporter	The person who found and reported the bug	Jane Smith (QA)
Date Reported	When the bug was found	2024-01-15
Module/Feature	Which part of the application is affected	Checkout / Coupon System
Environment	Browser, OS, device, app version, test environment details	Chrome 120, Windows 11, Staging environment v2.4.1
Preconditions	State the system must be in before the bug can be reproduced	User is logged in. Cart contains at least one item. A valid percentage coupon code "SAVE20" (20% off) exists in the system.
Steps to Reproduce	Exact, numbered steps another person can follow to see the bug	1. Log in as testuser@example.com. 2. Add any product to the cart. 3. Navigate to the Checkout page. 4. Enter "SAVE20" in the coupon field. 5. Click "Apply Coupon."

Field	Description	Example
Expected Result	What should happen according to the requirements	The coupon is applied. The total price is reduced by 20%. A success message "Coupon applied!" is displayed.
Actual Result	What actually happened	A server error (HTTP 500) is displayed. The coupon is not applied. The page shows "Something went wrong. Please try again."
Attachments	Screenshots, screen recordings, logs, error messages	Screenshot_error500.png, browser_console_log.txt
Reproducibility	How often the bug occurs: Always, Intermittent, One-time	Always (100% reproducible)
Assigned To	The developer responsible for fixing it	<i>(Assigned by lead/PM)</i>
Status	Current state of the bug in the defect lifecycle	New

The Golden Rule of Bug Reporting

The Golden Rule

A good bug report enables a developer who has never seen the bug to reproduce it on the first attempt, without asking any follow-up questions.

If a developer has to come back to you and ask "What did you mean by...?" or "How did you get to that screen?" or "What data were you using?", your bug report is incomplete.

Common Mistakes in Bug Reporting

Mistake	Bad Example	Good Example
Vague title	"Button not working"	"'Apply Coupon' button returns HTTP 500 error for percentage-based coupons on checkout page"
Missing steps	"Go to checkout and apply coupon. It doesn't work."	"1. Log in as testuser. 2. Add Product X to cart. 3. Go to Checkout. 4. Enter coupon 'SAVE20'. 5. Click 'Apply Coupon'."
Missing environment	<i>(not specified)</i>	"Chrome 120, Windows 11, Staging v2.4.1"
Vague expected result	"It should work"	"Total price should decrease by 20% and success message 'Coupon applied!' should display"
No screenshots	<i>(none attached)</i>	Screenshot of the error page, screenshot of the console showing the HTTP 500 error
Combining multiple bugs	"The coupon doesn't work AND the page layout is broken AND the back button doesn't work"	One bug report per defect, each with its own ID, steps, and tracking

5.5 Severity vs Priority

This is one of the most commonly confused pairs of concepts in software testing and one of the most frequently asked interview questions.

Definition

Severity measures the **technical impact** of a defect on the system — how badly the defect affects the software's functionality. Severity is determined by the **tester** based on the defect's technical impact.

Priority measures the **business urgency** of fixing the defect — how quickly it needs to be fixed from a business perspective. Priority is determined by the **product manager or project manager** based on business needs.

Why They Are Different

A defect can have high severity but low priority, or low severity but high priority. These are not the same dimension.

Scenario	Severity	Priority	Explanation
The application crashes when a user tries to make a payment	Critical	High	Major technical impact + critical business function = fix immediately
A typo in the company name on the homepage	Minor (cosmetically trivial)	High (brand reputation)	Technically insignificant, but publicly visible and embarrassing — fix quickly
An obscure feature used by 2 internal employees crashes	Critical (application crash)	Low	Severe technical impact, but affects almost nobody — schedule for a future release
A color on a rarely visited admin settings page is slightly wrong	Trivial	Low	Minor cosmetic issue on an unimportant page — fix whenever convenient

Severity Levels

Level	Description	Example
Critical	System crash, data loss, security breach, complete feature failure, no workaround	Application crashes on launch. Payment processing loses transaction data.
Major	Major feature is severely impaired but the system does not crash. A workaround may exist but is unacceptable for production.	Search returns completely wrong results. User cannot upload a profile picture.
Minor	Minor feature issue, cosmetic imperfection, or a problem with an easy workaround	A date displays in the wrong format (MM/DD/YYYY instead of DD/MM/YYYY).
Trivial	Cosmetic issue with no functional impact	An extra space between two words. A tooltip has a minor typo. Font size is 13px instead of 14px.

Priority Levels

Level	Description	Typical Action
High (P1)	Must be fixed immediately — blocks testing, blocks release, or has critical business impact	Fix in the current sprint/hotfix. Drop other work if necessary.
Medium (P2)	Should be fixed soon but is not an emergency — does not block testing or release	Fix in the current or next sprint.
Low (P3)	Can be fixed when convenient — minor issue with minimal business impact	Schedule for a future release when time permits.

Interview Tip

When asked "Can severity and priority be different?" in an interview, always say YES and give the classic example: "A typo in the company name on the login page is low severity (just a cosmetic text error) but high priority (it's the first thing users see and affects brand perception)." Then give the reverse: "A crash in an obscure admin feature used by only one person is high severity (crash) but low priority (affects almost no one)."

5.6 The Defect Life Cycle

What Is the Defect Life Cycle?

Definition

The **Defect Life Cycle** (also called the Bug Life Cycle) is the journey a defect takes from the moment it is discovered to the moment it is finally closed. It defines a series of states (statuses) and transitions that ensure every defect is systematically tracked, investigated, resolved, verified, and documented.

The Standard Defect Life Cycle States

stateDiagram-v2

[*] --> New: Bug found and reported

New --> Assigned: Lead assigns to developer

Assigned --> Open: Developer starts investigating

Open --> Fixed: Developer fixes the bug

Open --> Rejected: Not a valid bug

Open --> Deferred: Valid but fix postponed

Open --> Duplicate: Already reported

Fixed --> Retest: Tester re-verifies the fix

Retest --> Closed: Fix confirmed working

Retest --> Reopened: Fix did not work

Reopened --> Assigned: Reassigned to developer

Rejected --> Closed: Confirmed not a bug

Deferred --> Open: Picked up in future sprint

Duplicate --> Closed: Linked to original bug

Detailed Explanation of Each State

State	Who Acts	What Happens
New	Tester	The tester discovers a defect and logs it in the bug tracking tool with all required information (title, steps, severity, etc.). The bug is in "New" status.
Assigned	Test Lead / PM	The test lead or project manager reviews the bug report and assigns it to the appropriate developer for investigation.
Open	Developer	The developer acknowledges the bug and begins investigating. They reproduce the bug and analyze the root cause.
Fixed	Developer	The developer implements a code fix, commits it, and marks the bug as "Fixed." A new build is created containing the fix.
Retest	Tester	The tester re-executes the original test steps (and potentially additional related test cases) on the new build to verify that the fix works correctly and has not introduced new problems.
Closed	Tester	The tester confirms that the fix works correctly. The bug is marked as "Closed" — it has been successfully resolved.

State	Who Acts	What Happens
Reopened	Tester	The tester finds that the fix did NOT work — the bug still occurs, or the fix introduced a new problem. The bug is sent back to the developer.
Rejected	Developer	The developer determines that the reported behavior is not actually a bug — it is working as designed, or the tester misunderstood the requirement. The tester and developer discuss; if confirmed not a bug, it is closed.
Deferred	PM / Lead	The bug is valid, but the team decides to postpone fixing it — perhaps it affects a low-priority feature, or there is not enough time in the current sprint. It will be addressed in a future release.
Duplicate	Developer / Lead	The bug has already been reported by someone else. This report is linked to the original and closed as a duplicate.

Real-World Workflow Example

1. A tester at an e-commerce company discovers that the "Apply Coupon" button causes a server error. They log it as a New bug (BUG-2847).
2. The QA Lead reviews the report and assigns it to Developer Alex. Status: Assigned.
3. Alex opens the bug, reproduces it, and finds the root cause (a null pointer exception when the coupon percentage is a decimal). Status: Open.
4. Alex writes a fix, adds a unit test to prevent regression, commits the code, and marks the bug as Fixed.
5. The CI/CD pipeline builds a new version with the fix. The tester deploys it to the staging environment.
6. The tester re-executes the original test steps. The coupon now applies correctly. The tester also tests related scenarios (different coupon types, expired coupons, combined coupons). All pass. Status: Closed.

Alternatively, at step 6, if the fix did not work or introduced a new issue, the tester would Reopen the bug and it would cycle back through the process.

Industry Best Practice

In Agile teams, the full defect lifecycle — from New to Closed — should complete within a single sprint whenever possible. Bugs that spill across multiple sprints increase risk and complexity. High-performing teams aim for a "zero open bugs" policy at the end of each sprint, though pragmatically, some deferred bugs are carried in a backlog with clear prioritization.

5.7 Hour 5 Interview Corner

Q1: What are the components of a good test case? Model Answer: A well-structured test case includes: Test Case ID (unique identifier), Title (concise description), Preconditions (what must be true before execution), Test Data (specific input values), Test Steps (numbered, precise actions), Expected Result (specific, verifiable outcome), and fields for Actual Result and Status (filled during execution). Additional fields may include Priority, Module, and Post-conditions.

Q2: What is the difference between Severity and Priority? Can you give an example where they differ? Model Answer: Severity measures the technical impact of a defect on the system and is assessed by the tester. Priority measures the business urgency of fixing the defect and is determined by the product manager. They can differ: for example, a typo in the company name on the homepage has low severity (cosmetic text error) but high priority (publicly visible, affects brand reputation). Conversely, a crash in an obscure admin feature used by one internal employee has high severity (crash) but low priority (affects virtually no one).

Q3: Explain the Defect Life Cycle. Model Answer: The Defect Life Cycle defines the stages a bug goes through from discovery to resolution. A defect starts as "New" when found by a tester. It is then "Assigned" to a developer by the lead. The developer opens it and investigates. Once fixed, it moves to "Fixed" status. The tester then retests the fix. If it works, the bug is "Closed." If the fix did not work, the bug is "Reopened" and reassigned. Bugs can also be "Rejected" (not actually a bug), "Deferred" (valid but postponed), or "Duplicate" (already reported).

Q4: What makes a good bug report? Model Answer: A good bug report enables a developer to reproduce the bug on the first attempt without asking follow-up questions. It must include: a specific, searchable title; clear severity and priority; complete environment information; precise preconditions; exact numbered steps to reproduce; specific expected result versus actual result; screenshots or recordings; and reproducibility information (always, intermittent, one-time).

Q5: What is a Requirement Traceability Matrix? Why is it important? Model Answer: A Requirement Traceability Matrix (RTM) is a document that maps every requirement to its corresponding test cases and vice versa. It ensures complete coverage — every requirement has at least one test case — and identifies gaps where requirements are untested and redundancies where test cases do not map to any requirement. The RTM is a living document updated throughout the project and is critical for measuring test coverage and making informed release decisions.

Q6: What are the different states a bug can be in? Model Answer: The standard defect lifecycle states are: New (just reported), Assigned (given to a developer), Open (developer investigating), Fixed (developer has implemented a fix), Retest (tester verifying the fix), Closed (fix confirmed working), Reopened (fix did not work), Rejected (not a valid bug), Deferred (valid but postponed), and Duplicate (already reported elsewhere).

Consolidated Quick-Reference Glossary

Term	Definition
Software Testing	Process of evaluating software to find differences between expected and actual behavior
Quality Assurance (QA)	Process-oriented discipline focused on preventing defects through standards and processes
Quality Control (QC)	Product-oriented discipline focused on detecting defects in the product
Defect / Bug	A flaw in the software that causes it to behave differently from the specified requirements
SDLC	Software Development Life Cycle — the overall framework for developing software
STLC	Software Testing Life Cycle — the framework specifically for testing activities
Agile	An iterative, incremental approach to software development with short cycles called sprints

Term	Definition
Sprint	A short, time-boxed development cycle (typically 1–4 weeks) in Agile methodology
Entry Criteria	Conditions that must be met before a testing phase can begin
Exit Criteria	Conditions that must be met before testing is considered complete
RTM	Requirement Traceability Matrix — maps requirements to test cases
Smoke Testing	Quick, broad, shallow testing of core features to verify build stability
Sanity Testing	Focused, narrow, deep testing of a specific change or fix
Regression Testing	Re-running existing tests after changes to ensure nothing is broken
UAT	User Acceptance Testing — final testing by end users to validate business needs
Black-Box Testing	Testing without knowledge of internal code, based on specifications
White-Box Testing	Testing with full knowledge of internal code, based on code structure
Equivalence Partitioning	Dividing inputs into groups treated identically, testing one value per group
Boundary Value Analysis	Testing values at the edges of equivalence partitions where bugs cluster
Decision Table Testing	Systematically testing all combinations of multiple conditions
State Transition Testing	Testing behavior changes based on the system's current state and triggering events
Severity	Technical impact of a defect (Critical, Major, Minor, Trivial) — assessed by tester
Priority	Business urgency of fixing a defect (High, Medium, Low) — set by PM

Term	Definition
Defect Life Cycle	The sequence of states a bug passes through from discovery to closure
Shift-Left Testing	Starting testing activities as early as possible in the development lifecycle
CI/CD	Continuous Integration / Continuous Delivery — automated build and deployment pipelines

What Comes Next

In Hours 6–8, you will apply these theoretical concepts in practice using Postman for API testing. In Hours 9–11, you will use JMeter for performance testing. In Hours 12–15, you will automate web UI testing using Selenium with Java. Hour 16 brings everything together in a capstone mini-project. The theoretical foundation you have built in these first 5 hours will make every practical session more meaningful — you will understand not just *how* to use the tools, but *why* you are using them and *what* you are testing.

End of Hours 1–5 Theoretical Study Material